

MINI-PROJECT REPORT

Protocol Analysis with Wireshark

Unit 2: Network Models and Protocols

Student Name: Kotame Pranav **PRN:** 2125UCSM1061

1. Aim

To capture and analyze network packets using Wireshark, identify the TCP three-way handshake (SYN, SYN-ACK, ACK), and observe the subsequent HTTP GET request.

2. Software and Requirements

- Kali Linux
- Wireshark (open-source network protocol analyzer)
- Mozilla Firefox or another web browser
- Internet connection
- Active network interface: eth0

3. Procedure

1. Open Wireshark in Kali Linux and select the active network interface (eth0).
2. Start packet capture.
3. Open a web browser and navigate to <http://neverssl.com>.
4. Wait for the webpage to load, then stop the packet capture.
5. Apply the display filter 'http' to locate HTTP traffic.
6. Identify the HTTP GET request and its corresponding TCP connection.
7. Analyze the TCP three-way handshake: SYN, SYN-ACK, and ACK.
8. Record the source and destination IP addresses and TCP port numbers.
9. Save the packet capture in .pcapng format.

4. TCP Three-Way Handshake Observation

The captured TCP connection shows the standard three-way handshake used to establish a reliable TCP connection before HTTP data is exchanged.

Packet	Packet No.	Source IP	Source Port	Destination IP	Destination Port
SYN	87	192.168.17.130	34078	34.223.124.45	80
SYN-ACK	103	34.223.124.45	80	192.168.17.130	34078
ACK	104	192.168.17.130	34078	34.223.124.45	80

5. HTTP GET Request

After the TCP connection was established, the client sent an HTTP GET request to the server. The observed request was:

GET / HTTP/1.1

The HTTP server responded successfully with an HTTP 200 OK response.

Packet	Source IP	Source Port	Destination IP	Destination Port
HTTP GET (105)	192.168.17.130	34078	34.223.124.45	80

6. Transport Layer Analysis

The transport-layer protocol used in this communication is TCP. The client used the ephemeral source port 34078, while the web server listened on destination port 80, the standard port for HTTP. The TCP handshake established the connection before the HTTP GET request was transmitted.

7. Result

The network traffic was successfully captured and analyzed using Wireshark. The TCP three-way handshake and the subsequent HTTP GET request were identified successfully. The source and destination IP addresses and TCP port numbers were recorded from the capture.

8. Conclusion

This practical demonstrated how Wireshark can be used to inspect network protocols at the packet level. The analysis showed that a TCP connection is established using SYN, SYN-ACK, and ACK packets, after which the client can send an HTTP GET request over TCP port 80. Wireshark provides useful visibility into IP addresses, transport-layer ports, TCP flags, and application-layer HTTP traffic.

9. Screenshots

The following screenshots should be inserted in this section in the final submission:

Figure 1: TCP three-way handshake and HTTP GET packet (Packet 87, 103, 104, and 105).

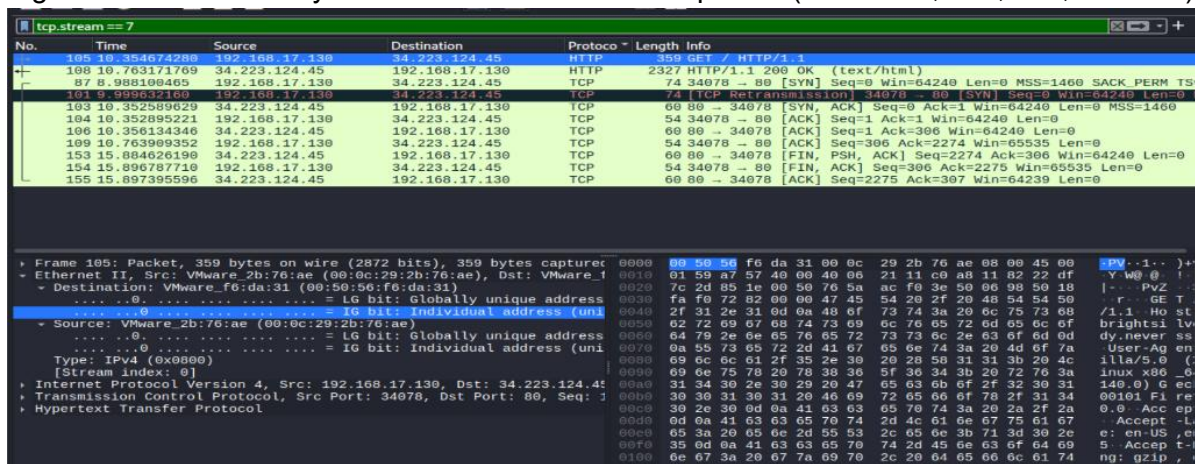
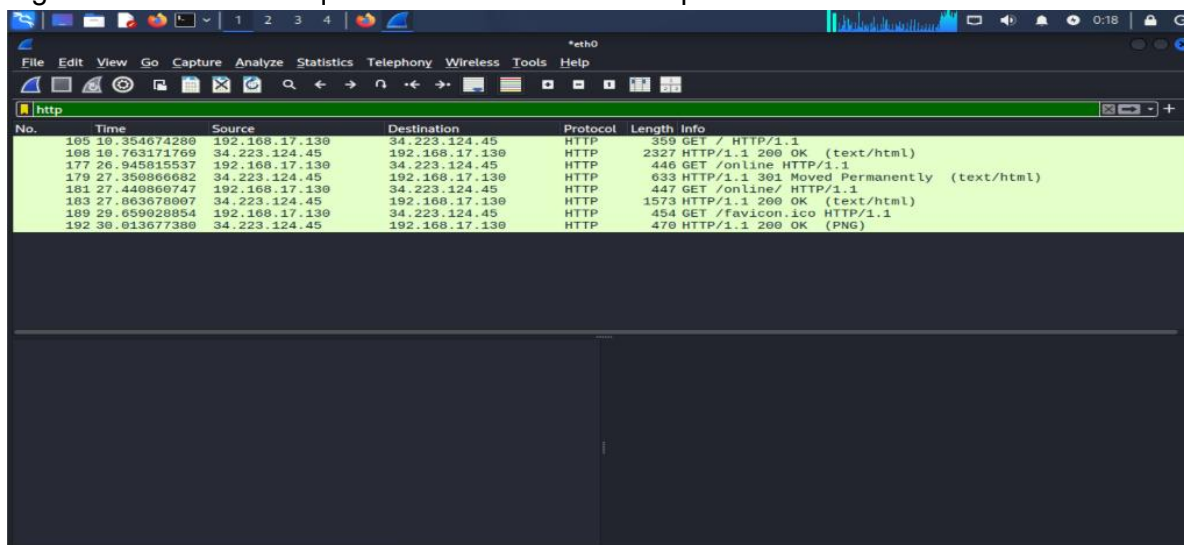


Figure 2: HTTP GET request and HTTP 200 OK response.



No.	Time	Source	Destination	Protocol	Length	Info
105	10.354674280	192.168.17.130	34.223.124.45	HTTP	399	GET / HTTP/1.1
108	10.763171769	34.223.124.45	192.168.17.130	HTTP	2327	HTTP/1.1 200 OK (text/html)
177	26.945815537	192.168.17.130	34.223.124.45	HTTP	446	GET /online HTTP/1.1
179	27.350866682	34.223.124.45	192.168.17.130	HTTP	633	HTTP/1.1 301 Moved Permanently (text/html)
181	27.440869747	192.168.17.130	34.223.124.45	HTTP	447	GET /online/ HTTP/1.1
183	27.863678907	34.223.124.45	192.168.17.130	HTTP	1573	HTTP/1.1 200 OK (text/html)
189	29.659028854	192.168.17.130	34.223.124.45	HTTP	454	GET /favicon.ico HTTP/1.1
192	30.013677380	34.223.124.45	192.168.17.130	HTTP	470	HTTP/1.1 200 OK (PNG)

10. GitHub Repository Link

The project report, Wireshark packet capture file (.pcapng), and screenshots are available on GitHub at:
<https://github.com/Pranavkotame/NPC-ASSIGNMENT-2.git>